

# Closing the Triage Loop

MCP-mediated KLEE validation of  
LLM-reported C/C++ vulnerabilities

Christopher Scherb, Alexander Trapp, Ruben Hutter,  
Nico Bachmann, Luc Bryan Heitz

University of Applied Sciences and Arts Northwestern Switzerland

**KES 2026** | **Dublin**

## curl: AI reports created real work for maintainers

---

< 5%

of security submissions  
confirmed as vulnerabilities  
in 2025

- ▶ Low-quality AI reports consumed maintainer time
- ▶ curl ended bounty rewards on **31 January 2026**
- ▶ By April, AI-assisted reports had improved in quality; volume was higher

### A convincing report still needs independent verification

Daniel Stenberg: The end of the curl bug-bounty (26 Jan 2026); High-Quality Chaos (22 Apr 2026).

# The problem: which AI findings are real?

---

**An LLM flags a possible C/C++ bug.  
Would you report it?**

- ▶ A convincing explanation is not execution evidence
- ▶ A failure may depend on inputs that real callers never provide
- ▶ Manually building a test and checking callers takes work

**Our goal: check each candidate and explain the conditions under which it can fail**

# The LLM selects what to check

---

- ▶ Read the code and identify a suspicious function
- ▶ Identify inputs that should vary during analysis
- ▶ Inspect callers and propose assumptions about valid inputs
- ▶ Submit a structured candidate to the verification tool

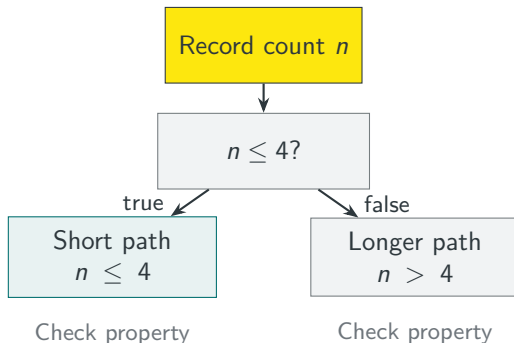


**The LLM supplies a hypothesis and context; execution must check it**

# Symbolic execution checks possible executions

---

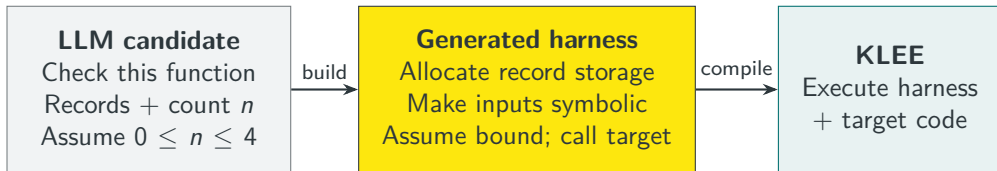
- ▶ Use variables instead of one fixed test input
- ▶ Follow branches and collect their conditions
- ▶ Solve those conditions to find possible paths
- ▶ If a path fails, obtain a concrete input that reaches it



**KLEE is the symbolic execution engine used in our system**

## The bridge: a small program KLEE can execute

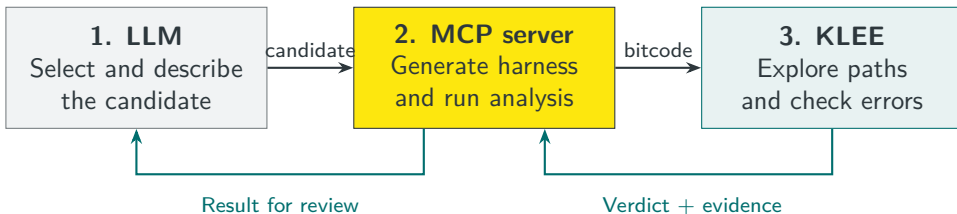
---



- ▶ A **harness** sets up inputs and calls the selected function
- ▶ The server turns declared inputs into symbolic variables
- ▶ Assumptions restrict those variables before the call
- ▶ The harness and target code are compiled to LLVM bitcode

# How the LLM and KLEE work together

---



- ▶ **LLM**: choose the candidate, inputs, and assumptions
- ▶ **MCP server**: generate the harness and manage KLEE runs
- ▶ **KLEE**: return a verdict and execution evidence
- ▶ **LLM**: explain the result and investigate caller context

# What the verdicts mean

---

|                   |                                                              |
|-------------------|--------------------------------------------------------------|
| <b>Confirmed</b>  | A matching error was found; a concrete input demonstrates it |
| <b>Infeasible</b> | No matching error exists in the fully explored input model   |
| <b>Unresolved</b> | The run timed out, failed to build, or hit an engine error   |

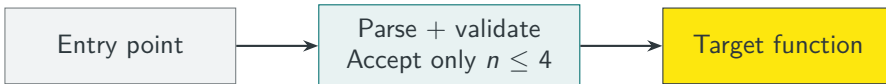
- ▶ **Confirmed:** check whether real callers can supply that input
- ▶ **Infeasible:** ask which inputs the assumptions excluded



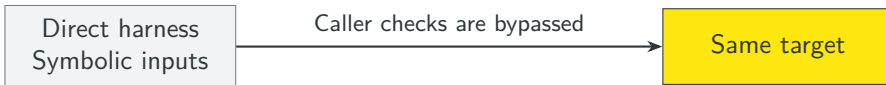
# Where do we start symbolic execution?

---

## START FROM AN APPLICATION / API ENTRY



## START DIRECTLY AT THE FUNCTION

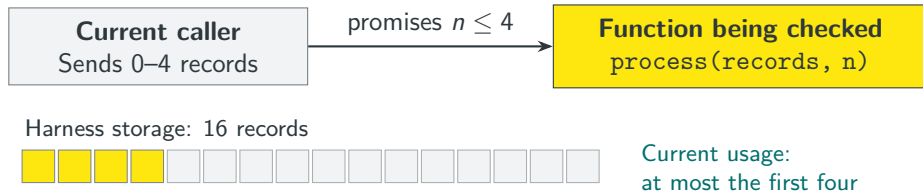


- ▶ **From an entry point:** preserve caller checks; often a larger search
- ▶ **Directly at the function:** focus the search, but lose caller guarantees

**Our direct harness needs assumptions about the missing caller context**

# A caller contract is a promise about inputs

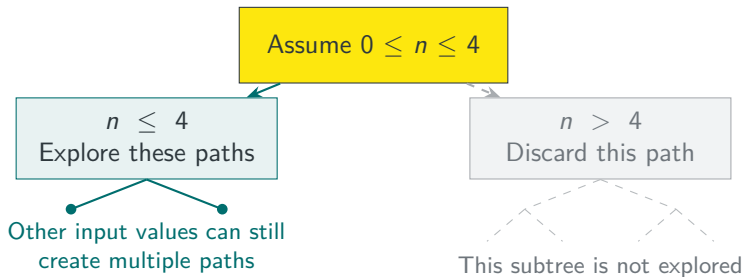
---



- ▶ **Caller contract:** “I will pass at most four records”
- ▶ A direct harness does not execute the caller that keeps this promise
- ▶ The LLM models this guarantee with the bound  $0 \leq n \leq 4$

**An assumption in the harness does not add a check to the real function**

# Bounds remove paths from the search

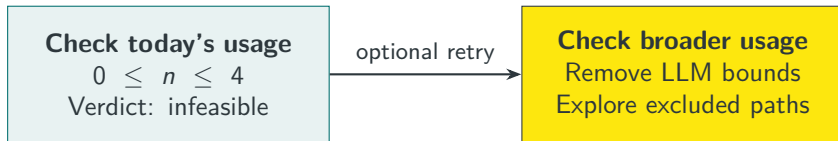


- ▶ Paths requiring  $n > 4$  conflict with the bound and are discarded
- ▶ In a loop over  $n$  records, at most four iterations are needed
- ▶ Fewer execution states usually mean less solver work and memory

**Less exploration also means a narrower safety claim**

# Why retry: what if the function is used differently?

---



**Possible results:** an error appears; no error is found after completion; or the larger search remains unresolved.

- ▶ The LLM may have inferred a bound that is too restrictive
- ▶ A new caller or refactor may pass more records, such as  $n = 8$
- ▶ Retry at the same function, without the LLM bounds

**A new error can reveal a fragile assumption, even if today's usage is safe**

# Proof-of-concept evaluation

---

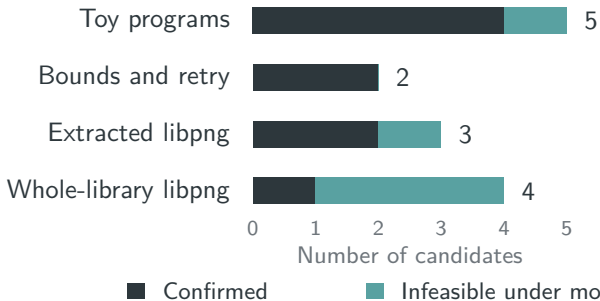
# 14

hand-constructed  
candidates

- ▶ Toy programs for selected weakness classes
  - ▶ Bounds and retry sensitivity cases
  - ▶ Extracted libpng functions
  - ▶ Whole-library libpng + zlib analysis
- 
- ▶ KLEE 3.1; LLVM/Clang 13; STP + MiniSat
  - ▶ Whole-library target: 2.2 MB bitcode, 30 translation units

# All 14 rows match their expected verdict

---



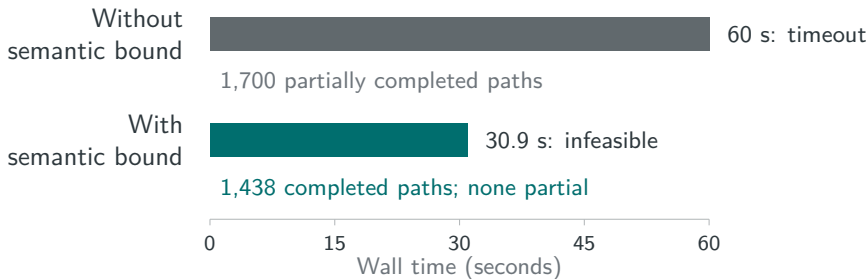
- ▶ 9 confirmed
- ▶ 5 infeasible
- ▶ 0.9–42.9 s per row

## Evidence of internal consistency on these selected cases

- ▶ External precision, recall, and analyst time savings remain unmeasured

# Bounds can make exploration finish

---

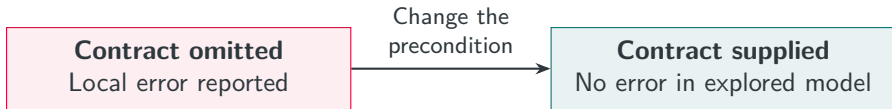


- ▶ Same libpng parser and harness; with and without a semantic bound
- ▶ The terminating result applies only under the added contract

# The same function can be safe under its contract

---

- ▶ CVE-2015-8540 case study: an internal libpng helper
- ▶ Function-level analysis reports an error across three versions
- ▶ Adding the caller contract changes the verdict to infeasible

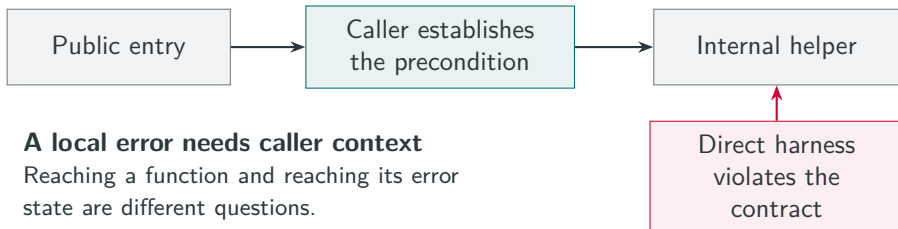


**Fixing the caller can make normal usage safe while the helper still trusts it**



# Can today's callers reach the error?

---



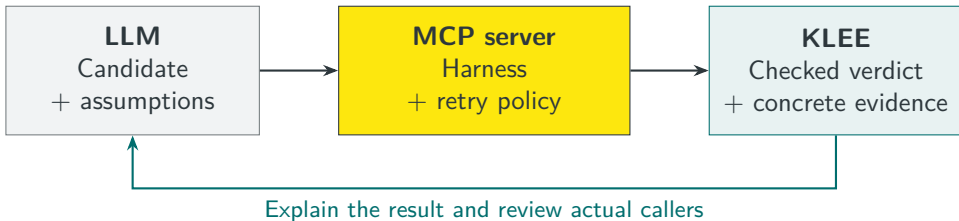
## **A local error needs caller context**

Reaching a function and reaching its error state are different questions.

- ▶ `png_format_number`: an unchecked internal precondition
- ▶ Shipped callers satisfy the precondition at every reviewed call site
- ▶ Hardening opportunity: guard the precondition against future misuse

## Conclusion: the LLM guides, KLEE checks

---



- ▶ Focus on a candidate and model relevant caller guarantees
- ▶ Use bounds to make that analysis tractable
- ▶ Retry to reveal behavior hidden by the assumptions

**Report the verdict together with the assumptions that made it possible**

# Would you report it now?

---

- ▶ **Evidence:** can an execution demonstrate the error?
- ▶ **Assumptions:** for which inputs was the function checked?
- ▶ **Usage:** can a current or future caller violate those assumptions?

## Check the finding. Explain its scope.

Questions?

[github.com/FHNW-Security-Lab/klee-mcp](https://github.com/FHNW-Security-Lab/klee-mcp)

## Backup: the benchmark rows

| Candidate             | Verdict                       | Wall time (s) |
|-----------------------|-------------------------------|---------------|
| bof_01                | confirmed                     | 1.0           |
| intoverflow_01        | confirmed                     | 2.4           |
| null_01               | confirmed                     | 0.9           |
| uaf_01                | confirmed                     | 0.9           |
| safe_01               | infeasible                    | 1.1           |
| bof_01_bounded        | confirmed                     | 0.9           |
| bof_01_tootight       | confirmed after retry         | 0.9           |
| libpng_fp_number      | infeasible                    | 30.9          |
| libpng_fp_number_bug  | confirmed                     | 5.9           |
| libpng_fp_number_unb. | confirmed                     | 9.4           |
| libpng_sig_cmp        | infeasible                    | 5.1           |
| libpng_fp_whole       | infeasible                    | 42.9          |
| libpng_push_read      | infeasible                    | 4.1           |
| libpng_format_number  | confirmed; contract violation | 4.2           |

Infeasible verdicts are conditional on the harness and engine model.

## Backup: larger targets expose model limits

---

| Target        | Run result                             | Interpretation                                             |
|---------------|----------------------------------------|------------------------------------------------------------|
| libpng tRNS   | 240 s; timeout                         | Analysis unresolved                                        |
| libjpeg-turbo | 5.4 s; infeasible<br>1 completed path  | Error handling limits exploration; weak evidence of safety |
| libxml2       | 23 s; infeasible<br>11 completed paths | A second whole-library run; still only the modeled inputs  |

- ▶ Whole-library linking is not whole-program coverage
- ▶ Read path statistics and unsupported behavior with the verdict

## Backup: limitations and next steps

---

- ▶ **Small benchmark:** evaluate an independent set of real findings
- ▶ **Engine limits:** improve handling of complex parser error paths
- ▶ **Caller assumptions:** validate them against realistic entry points
- ▶ **Impact assessment:** review automated labels before reporting

**A local failure is one step toward a useful security report**

## Backup: further reading

---

- ▶ Cadar, Dunbar, Engler (2008)  
*KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs*, OSDI
- ▶ Ramos and Engler (2015)  
*Under-Constrained Symbolic Execution: Correctness Checking for Real Code*, USENIX Security

Project: [github.com/FHNW-Security-Lab/klee-mcp](https://github.com/FHNW-Security-Lab/klee-mcp)